

Seeing the Same Task Differently

Vision-Language Model Operation, Benchmark Comparison,
and a Qwen2.5-VL Fine-Tuning Case Study

A report built from the benchmark and fine-tuning artifacts
stored in this repository

Primary analysis command:

```
python comparison\build_report.py -results-dir results -output-dir  
comparison\output -bootstrap-samples 5000
```

Abstract

This report starts from a practical question: if the same harness gives the same kinds of images and prompts to several vision-language models, what actually changes in their answers? The setup in this repository is a good place to ask that question. Whether a model is loaded locally through Hugging Face or called through an API wrapper, the outside contract is simple: an image and a prompt go in, text comes out, and the benchmark turns that text into a score plus some runtime information.

The main comparison covers 294 model–benchmark cells: fourteen models, twenty- one benchmarks, and four task families: captioning, visual question answering (QA), labeling, and detection. On the benchmark-normalized summary, `gpt-5.5` comes first at 0.825, with `gpt-4.1` second at 0.757. That is the headline result, but it is not enough by itself. `qwen2.5-vl-3b-instruct` has the most first-place finishes and is the strongest captioning model in this run. PaliGemma looks weak in the overall mean, yet leads the normalized QA summary. In practice, the right model depends heavily on the kind of answer the application needs.

The report also includes a small Fashion-MNIST fine-tuning case study. Balanced QLoRA adaptation of Qwen2.5-VL-3B improves held-out accuracy from 19/50 to 24/50. That is a real gain on this sample, but it is not a clean win: the adapted model predicts *shirt* much more often and loses ground on nearby clothing classes. The useful lesson is narrow but important. Fine-tuning can move a small model, but the error pattern has to be checked just as carefully as the headline accuracy.

How to read this report. Performance claims in the main comparison come from `comparison/output/`, regenerated from the top-level `results/` JSON files. Fine-tuning results are handled separately; they do not enter the aggregate leaderboard.

Contents

Abstract	ii
1 Research Questions and Scope	1
1.1 Why compare through one harness?	1
1.2 Questions addressed	1
1.3 Evidence boundary	2
2 Models	3
2.1 One contract, several paths	3
2.2 Model families, transformer structure, and paths	3
2.3 The common multimodal transformer pattern	4
2.4 API-served vision models	5
2.5 Local Hugging Face models	5
2.6 Qwen2.5-VL and adapter loading	6
2.7 LLaVA’s practical compatibility handling	7
2.8 PaliGemma and compatible processing	8
2.9 Where the architectures differ	8
3 Datasets and Benchmarks	10
3.1 Twenty-one benchmarks, four families	10
3.2 Coverage is broad enough to compare, not to generalize universally	10
3.3 Reading the figures together	11
4 Evaluation Methodology	12
4.1 The evaluation loop	12
4.2 What is recorded	12
4.3 A necessary warning about normalization	13
5 Baseline Results and Comparison	14
5.1 The leading aggregate result	14

5.2	Raw scores give context, not a replacement leaderboard	15
5.3	Winning often is not the same as winning broadly	15
5.4	Task-family comparison	16
5.4.1	Four families tell four different stories	16
5.4.2	Captioning: local Qwen and LLaVA are competitive for a reason	17
5.4.3	QA and labeling: an instructive reversal	18
5.4.4	Detection: low values and a non-discriminating task	19
5.5	Spread, ranks, and uncertainty	20
5.5.1	Which benchmarks separate models?	20
5.5.2	Ranks retain ties that averages obscure	20
5.5.3	Head-to-head comparisons are close at the top	20
5.5.4	Intervals make overstatement harder	20
5.6	Runtime and resource trade-offs	21
5.6.1	Recorded timing	21
5.6.2	A practical quality–latency frontier	22
5.6.3	Memory values require environmental context	22
6	Fine-Tuning Experiments	23
6.1	Why a separate case study?	23
6.2	QLoRA workflow	23
6.3	Accuracy improves, but the error distribution moves	23
6.4	Class-level improvement and class bias	24
6.5	The Colab smoke result	25
7	Limitations and Next Experiments	26
7.1	Limits of the present evidence	26
7.2	Next experiments worth running	26
	Reproduction Notes and Sources	27
	Reproducing the comparison	27
	Local sources used	27
	External architecture references	28
	Closing observation	28

Chapter 1

Research Questions and Scope

1.1 Why compare through one harness?

Vision-language models are usually introduced through parameter counts, architecture names, or public benchmark scores. Those are useful facts, but they do not answer the question that matters inside this repository: given a PIL image, a prompt, and an expected output format, which wrapper behaves best when it is actually used in this benchmark code?

That kind of question needs a fairly ordinary kind of care. The prompts have to ask for a parseable answer. Predictions have to be normalized in the same way. Captioning scores and classification accuracies cannot simply be averaged as if they were the same unit. Runtime numbers need to be kept, but they also need context, because a hosted API call and a local GPU run are not the same experiment.

The repository gives us a clean comparison point through a small shared interface. Every model wrapper exposes `predict(image, prompt)`: image and prompt in, text out. Every benchmark builds a prompt, sends it to the wrapper, parses the answer, and records a score. That is the level this report trusts. It compares what the wrappers do in this harness, not what a model name may suggest in isolation.

1.2 Questions addressed

I organize the analysis around three questions.

1. Which of the evaluated models is strongest overall, after respecting differences between benchmark metrics?
2. Do particular models have credible task specialisms, even when they do not top the aggregate table?
3. In a small, held-out clothing classification experiment, does QLoRA adaptation improve Qwen2.5-VL-3B, and what new mistakes appear?

The first two questions use the main comparison output: fourteen model labels, twenty-one benchmarks, and four task families. The fine-tuning question uses separate nested result files. Once a model has been adapted, it is no longer part of the same baseline ranking, so those results are kept out of the general leaderboard.

1.3 Evidence boundary

The model labels in the figures are the result identifiers used by this repository. For local models, we can discuss loading, processing, and decoding because the wrapper code is present. For API-served models, the evidence is more limited: the wrapper submits text plus a data-URL image and receives text back. The report therefore does not infer private architecture from an API model name.

Table 1.1: Evidence layers used in the report.

Layer	Evidence	What it supports
Implementation	models/, benchmarks/, fine-tuning/	Prompt, input, generation, parsing, and adaptation workflow statements.
Main results	CSV tables and PNG plots in comparison/output/	Aggregate, family, benchmark, pairwise, uncertainty, and telemetry claims.
Adapter case study	JSON reports in results/fine-tuning/ and the downloaded smoke artifact folder	Paired Fashion-MNIST outcomes only.

Chapter 2

Models

2.1 One contract, several paths

From the benchmark’s point of view, every model looks similar: it receives an image and a prompt, then returns text. Inside the wrappers, the paths are very different. This chapter is about those visible paths in the repository. It is not trying to reconstruct private model internals or treat a model family name as a complete architectural description.

2.2 Model families, transformer structure, and paths

There are two useful identifiers for each model: the wrapper path we can read, and the checkpoint or API model ID that the wrapper calls. For hosted API models, the wrapper is the honest boundary of the evidence. For local checkpoints, the common pattern is easier to describe: visual information is turned into embeddings or image tokens, then passed into a language model that generates the answer.

Table 2.1: Evaluated model labels covered by the architecture discussion.

Architecture family	Model labels in the regenerated comparison
OpenAI API vision pathway	gpt-4.1, gpt-4o, gpt-5, gpt-5.1, gpt-5.2, gpt-5.3-chat-latest, gpt-5.4, gpt-5.4-mini, gpt-5.4-nano, gpt-5.5.
Qwen2.5-VL	qwen2.5-vl-3b-instruct.
LLaVA-style local models	llava-1.5-7b-hf, llava-gemma-2b.
PaliGemma	paligemma-3b-mix-224.

Table 2.2: Model paths and architecture level used in the report.

Family	Repository path	Model ID or checkpoint path	Architecture described here
OpenAI vision models	models/_openai_vision.py; wrappers such as models/gpt_5_5.py	gpt-4.1, gpt-4o, gpt-5.* API model IDs	Hosted image-to-text transformer systems observed only through the Responses API request and returned text.
Qwen2.5-VL	models/_qwen2_5_vl.py; models/qwen2_5_vl_3b_instruct.py	Qwen/Qwen2.5-VL-3B-Instruct	Vision-language transformer using a Qwen processor, image inputs, chat template, and autoregressive text decoding; optional PEFT adapter loading.
LLaVA	models/llava15_7b.py	llava-hf/llava-1.5-7b-hf	CLIP-style visual tokens aligned with a causal language model through image-token placeholders and checkpoint-specific processor settings.
PaliGemma	models/paligemma_3b_mix_224.py	google/paligemma-3b-mix-224	PaLI/Gemma-style vision-language conditional generation with paired image and text processing before decoder generation.

Path	Processing sequence visible in code	Output
API vision wrapper	Convert image to RGB JPEG data URL; pair <code>input_text</code> with <code>input_image</code> ; request a response from the configured model ID.	Response output text
Local generic wrapper	Apply processor chat template to an image/text turn; build tensors; generate tokens on the loaded model; decode only generated continuation.	Decoded continuation
Qwen2.5-VL	Apply Qwen chat template and image processor; optionally attach a PEFT adapter before generation.	Decoded continuation
LLaVA	Prepare image-token prompt; synchronize image-token configuration when necessary; generate and clean task-shaped text.	Cleaned task answer

Figure 2.1: Operational paths exposed by the repository wrappers.

2.3 The common multimodal transformer pattern

The local open models in this report all use the same broad recipe. An image is split into patches or otherwise converted into visual tokens. A vision transformer turns those tokens into hidden states. A small connector or model-specific merger maps the visual states into the language model’s token space. The language model is then used as an autoregressive decoder: it sees the prompt plus visual context and generates answer tokens one at a time.

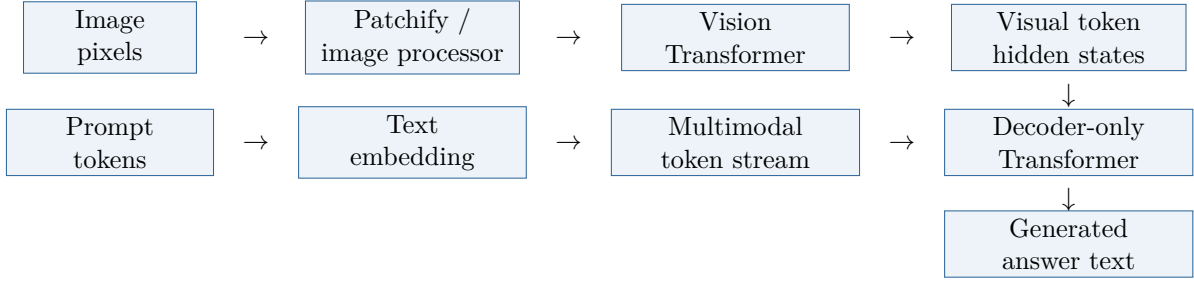


Figure 2.2: Common architecture pattern for local vision-language transformers: visual tokens are projected into a language-token stream before autoregressive decoding.

The differences between Qwen2.5-VL, LLaVA, and PaliGemma mostly lie in three places: the visual encoder, the connector between vision and language, and the language backbone. Those choices affect how well a model handles detail, layout, long prompts, and task-specific output formatting.

2.4 API-served vision models

The API pathway lives in `models/_openai_vision.py`. It does very little on purpose: convert the PIL image to RGB, encode it as a JPEG data URL, send it with the cleaned prompt through the Responses API, and return the output text. The GPT-5 wrapper variant also avoids sending temperature where that API configuration does not support it.

That gives the benchmark a convenient interface, but it also limits what we can claim. We can compare answer quality and observed request timing. We cannot inspect token-level visual features or provider-side memory use. So when the report compares API labels with local labels, it is comparing their end-to-end behavior in this harness, not running a controlled hardware or architecture ablation.

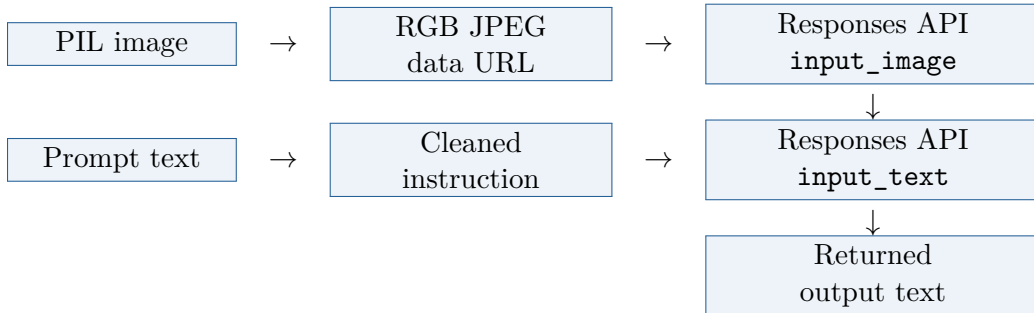


Figure 2.3: Observable architecture boundary for API-served model labels. The repo can describe the request and response path, but not the provider-side transformer internals.

2.5 Local Hugging Face models

The local wrappers share loading code from `models/_hf_model.py`. They try the local cache first and fall back to the hub when needed. The generic image-text route builds a one-image, one-instruction conversation, applies the processor’s chat template, moves tensors to the model device, generates a continuation, and decodes only the new tokens.

This route is useful because we can read and adjust the exact prompt preparation, generation settings, and local telemetry. The trade-off is practical: local hardware decides which checkpoints

are feasible and how slow the run will be.

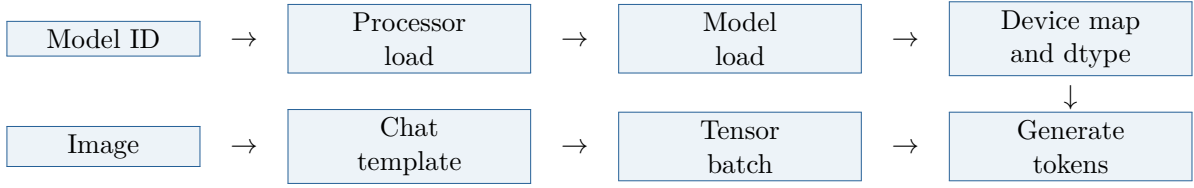


Figure 2.4: Generic Hugging Face execution graph used by the local wrappers: processor and model loading are separate from the per-example generation path.

2.6 Qwen2.5-VL and adapter loading

The Qwen2.5-VL wrapper uses `Qwen2_5_VLForConditionalGeneration`. It builds a user conversation with an image placeholder and the benchmark prompt, lets the processor prepare the text and image inputs, generates a bounded continuation, and decodes the answer. If an adapter path is provided, the wrapper attaches a PEFT adapter before evaluation.

That adapter hook matters because it lets the report move from comparison to intervention. The baseline runs ask how the model behaves as-is. The Fashion-MNIST case study asks whether a small task-specific update changes that behavior, while keeping the benchmark runner and answer parsing the same.

Architecturally, Qwen2.5-VL follows a three-part vision-language transformer design. The image side uses a native dynamic-resolution Vision Transformer, so the processor can preserve more of the image’s original spatial structure instead of forcing every input into one fixed token pattern. Window attention keeps the vision encoder more efficient on high-resolution inputs. A multimodal merger then maps visual representations into the Qwen language model’s embedding space. The decoder is a Qwen transformer language backbone, with multimodal rotary position handling used to keep visual and textual positions aligned.

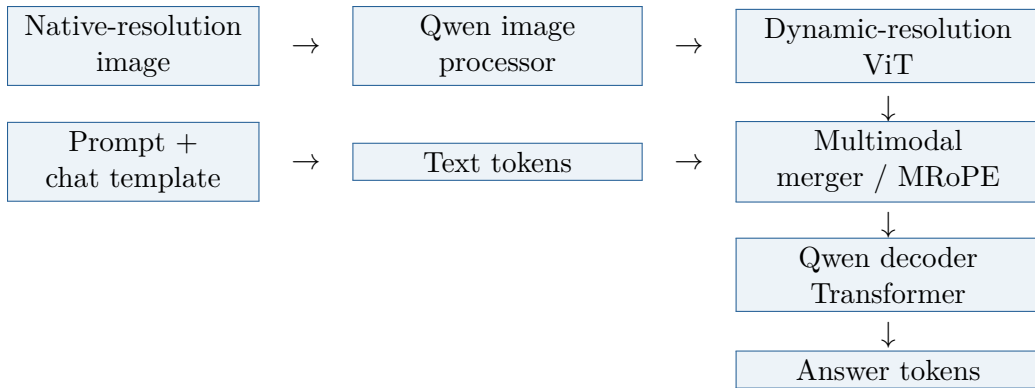


Figure 2.5: Qwen2.5-VL architecture as used by the local wrapper: a dynamic-resolution ViT feeds a multimodal token stream consumed by the Qwen decoder.

The adapter path adds a small trainable layer on top of that base architecture. In the fine-tuning experiment, LoRA modules are attached to attention projection matrices. During evaluation the benchmark still sees the same `predict(image, prompt)` interface; the difference is that the Qwen decoder now contains the loaded PEFT adapter.

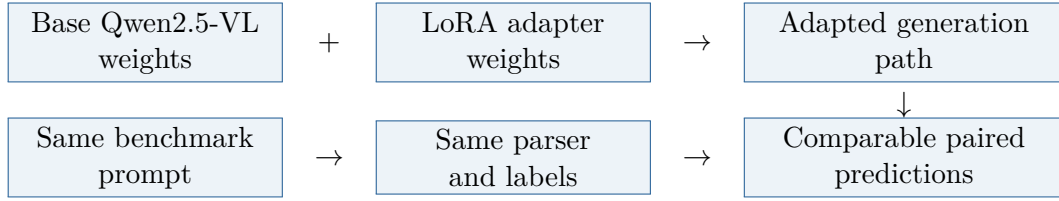


Figure 2.6: Adapter loading graph for the Qwen fine-tuning case study. The model changes, but the benchmark input and scoring path stay fixed.

2.7 LLaVA’s practical compatibility handling

The LLaVA wrapper is a reminder that a checkpoint ID is not enough. Some LLaVA-style checkpoints need the processor settings to match the visual token sequence expected by the model. The wrapper reads the visual patch settings and repairs image-token mismatches when needed. It also cleans the generated text differently by task: a classification answer is reduced to one line, while detection output has to keep parseable labelled boxes.

This is not just housekeeping. A bad prompt template or image-token mismatch can make a capable checkpoint look broken. The wrapper is therefore part of the experiment, not just plumbing around it.

LLaVA-1.5 is a deliberately simple multimodal transformer stack. A CLIP Vision Transformer encodes the image into patch-level visual states. A vision-language connector, implemented as an MLP in LLaVA-1.5, projects those visual states into the language model’s embedding space. The prompt contains an image placeholder, and the projected visual tokens occupy that position in the sequence seen by the language model. The answer is generated by an autoregressive decoder language model.

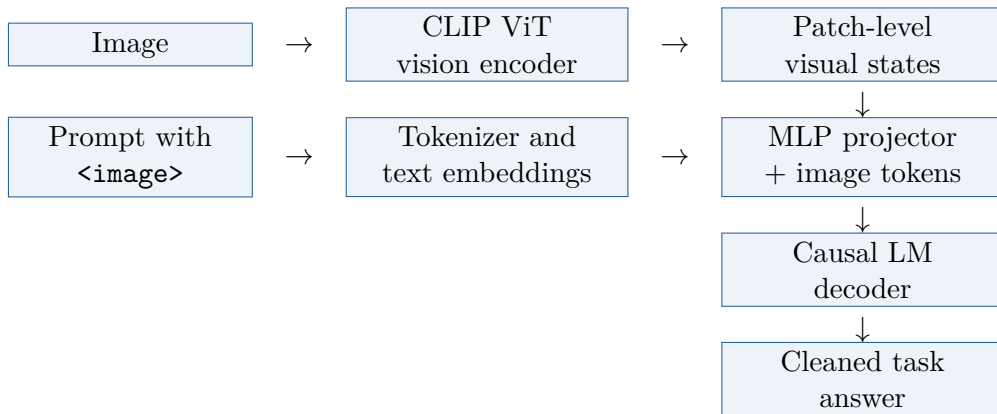


Figure 2.7: LLaVA-style architecture: a CLIP vision transformer is connected to a causal language model through a learned projector.

The evaluated `llava-gemma-2b` label follows the same broad idea—vision tokens connected to a decoder language model—but with a different language backbone and checkpoint configuration. That is why the report treats it as a LLaVA-style local model while still relying on the wrapper’s compatibility logic for the exact processor settings.

2.8 PaliGemma and compatible processing

The PaliGemma wrapper chooses the available compatible conditional-generation class, processes the image and prompt together, and decodes the generated tokens. The path is simpler than the LLaVA repair logic, but the results are one of the more useful reminders in the report: a model can be weak in the aggregate table and still be competitive for a particular family, in this case QA.

PaliGemma is built from open components in the PaLI/Gemma line: a SigLIP vision encoder and a Gemma language model joined for conditional generation. Compared with the LLaVA wrapper, the local code path is less about repairing an explicit `<image>` placeholder and more about giving the processor the image and text together so the model can form the combined sequence expected by the checkpoint.

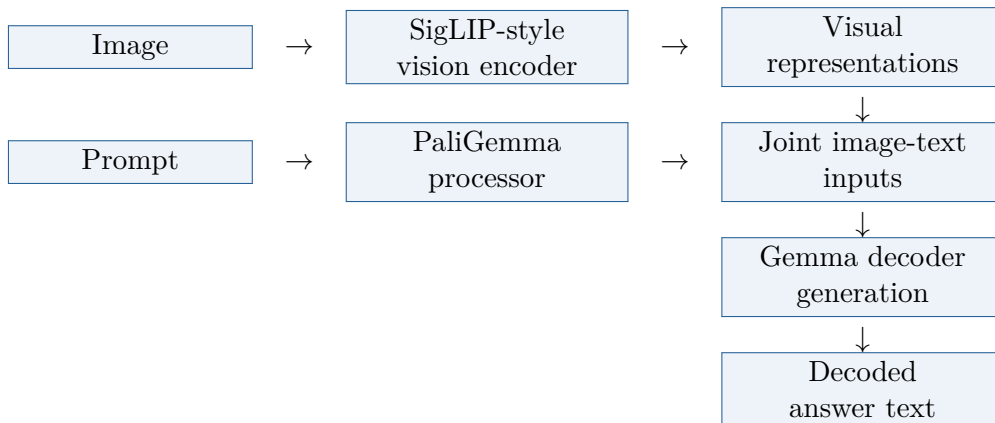


Figure 2.8: PaliGemma architecture: SigLIP-style visual representations are paired with a Gemma language decoder for image-conditioned generation.

2.9 Where the architectures differ

The practical differences between these models can be summarized by where vision enters the transformer computation. In the API-served path, the repository sees only the boundary. In Qwen2.5-VL, the model-specific processor and merger create a native multimodal sequence for the Qwen decoder. In LLaVA, the connector explicitly projects CLIP visual states into the language embedding space. In PaliGemma, the processor builds the paired image-text inputs expected by the SigLIP-plus-Gemma conditional-generation stack.

Family	Vision side	Fusion point	Language side
OpenAI API path	Not inspectable in this repository.	Provider-side, not visible to the harness.	Returned text only.
Qwen2.5-VL	Dynamic-resolution ViT with efficient visual attention.	Model-specific multimodal merger and position handling.	Qwen decoder transformer.
LLaVA	CLIP Vision Transformer visual states.	MLP projector inserts visual tokens into the language stream.	Causal language model decoder.
PaliGemma	SigLIP-style vision encoder.	Processor/model build paired image-text inputs.	Gemma decoder transformer.

Figure 2.9: Comparison graph for the main architecture families. The central difference is the fusion point between vision features and language decoding.

Operational conclusion. The evaluation treats each system as an image-to-text component. The wrappers handle the family-specific details needed to avoid obvious formatting failures. The results are therefore best read as performance of a model *plus its wrapper and prompt protocol*.

Chapter 3

Datasets and Benchmarks

3.1 Twenty-one benchmarks, four families

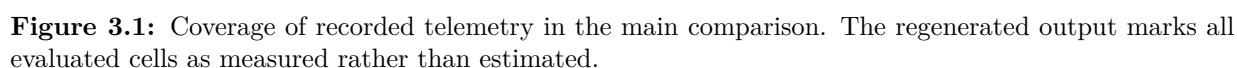
The aggregate run contains twenty-one benchmarks, grouped into captioning, QA, labeling, and detection. These families ask for different kinds of text, so they reward different habits in a model. Captioning needs a short description. QA needs a concise answer to a visual question. Labeling asks the model to choose from a class list. Detection asks for both a label and usable coordinates.

Table 3.1: Task families represented in the current aggregate comparison.

Family	Examples appearing in output	What the model must return
Captioning	flickr30k, textcaps, mscoco_caption, conceptual_captions_caption	A short natural-language description.
QA	docvqa, gqa, vqav2	A question-conditioned answer string.
Labeling	fashion_mnist, imagenet1k, places, inaturalist, lsun, mvtec_ad, video/image label tasks	One label from a candidate list.
Detection	mscoco, openimages_v4_detection, flickr30k_entities	Labels paired with parseable boxes.

3.2 Coverage is broad enough to compare, not to generalize universally

Each of the fourteen model labels appears on each of the twenty-one current benchmarks, giving a rectangular 294-cell comparison. That is important: the aggregate table is not inflated by one model being evaluated only on its best tasks. Still, this is not a universal ranking of multimodal systems. Some task runs have modest sample sizes, and several benchmarks produce large ties.



The results section uses three views repeatedly. The normalized score plot is the shortest answer to “who did well across these mixed tasks?” The raw score plot keeps the original metric scale visible, but should not carry the whole comparison by itself. Heatmaps and family plots show where the averages come from. When these views disagree, that disagreement is usually the point, not a problem to smooth away.

Chapter 4

Evaluation Methodology

4.1 The evaluation loop

The central pipeline is deliberately plain. A dataset provides a row and an image. A benchmark turns that row into a constrained prompt. A model wrapper runs its own inference path and returns text. The benchmark parses that text, records whether it was useful, and stores telemetry. The comparison script then reads the JSON reports and builds the tables and plots used here.

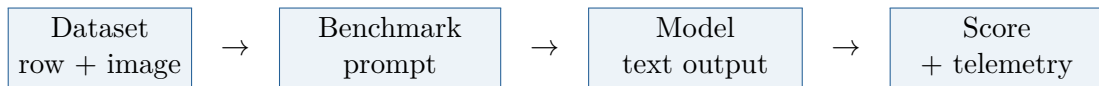


Figure 4.1: The observable evaluation pipeline shared by the model wrappers.

For labeling tasks, the prompt asks for exactly one label from a supplied list. The benchmark normalizes the model’s text and counts it as correct if it matches one of the valid labels for that row. When the label set is large, the benchmark keeps the valid labels and samples distractors with a row-derived seed. That keeps the choice set stable rather than adding avoidable noise.

Captioning, QA, and detection keep the same input/output shape but use different scoring logic. Captioning scores caption text. QA scores answer text. Detection parses labelled coordinate expressions. Some video-oriented datasets are represented as contact sheets of frames, which keeps the model contract image-based while still giving the model temporal clues.

4.2 What is recorded

Each JSON report contains per-example outputs and aggregate statistics. The comparison builder extracts the primary score, result count, wall-clock time, generation time, memory peaks, and whether telemetry was measured. In the regenerated table used here, all 294 main cells have measured telemetry; none are marked as estimated.

Table 4.1: Scoring concepts used in the comparison builder.

Quantity	Meaning in this report
Raw primary score	Original score for a benchmark, useful within its task type but not automatically commensurate across families.
Within-benchmark normalized score	For each benchmark, a model’s position between the observed minimum and maximum; used for the overall ranking.
Benchmark rank	Rank of each model on a benchmark, with tied maxima receiving the same first rank.
Pairwise win rate	Fraction of shared benchmarks on which model A beats model B, with ties contributing half in the generated summary.
Bootstrap interval	Variation in mean raw score under benchmark resampling; used as a caution about ordering stability.

4.3 A necessary warning about normalization

Within-benchmark normalization asks a useful question: compared with the other models on this same task, how strong was this system? That prevents metrics with naturally larger numbers from dominating the aggregate. The trade-off is that a small absolute difference on a tightly clustered benchmark can look more important than it really is, while an all-tie benchmark adds little evidence. For that reason, the report keeps normalized rankings next to raw plots, spreads, ranks, and family-level discussion.

Baseline Results and Comparison

5.1 The leading aggregate result

The normalized comparison has a clear leader: **gpt-5.5** reaches 0.825 with a mean benchmark rank of 2.76. **gpt-4.1** follows at 0.757, a gap of 0.068 normalized points. Behind them, **gpt-5.1**, **gpt-5.4-mini**, **gpt-5.4**, and **qwen2.5-vl-3b-instruct** sit in a fairly tight group from 0.742 down to 0.714. I would not read every adjacent rank in that group as a decisive practical difference.

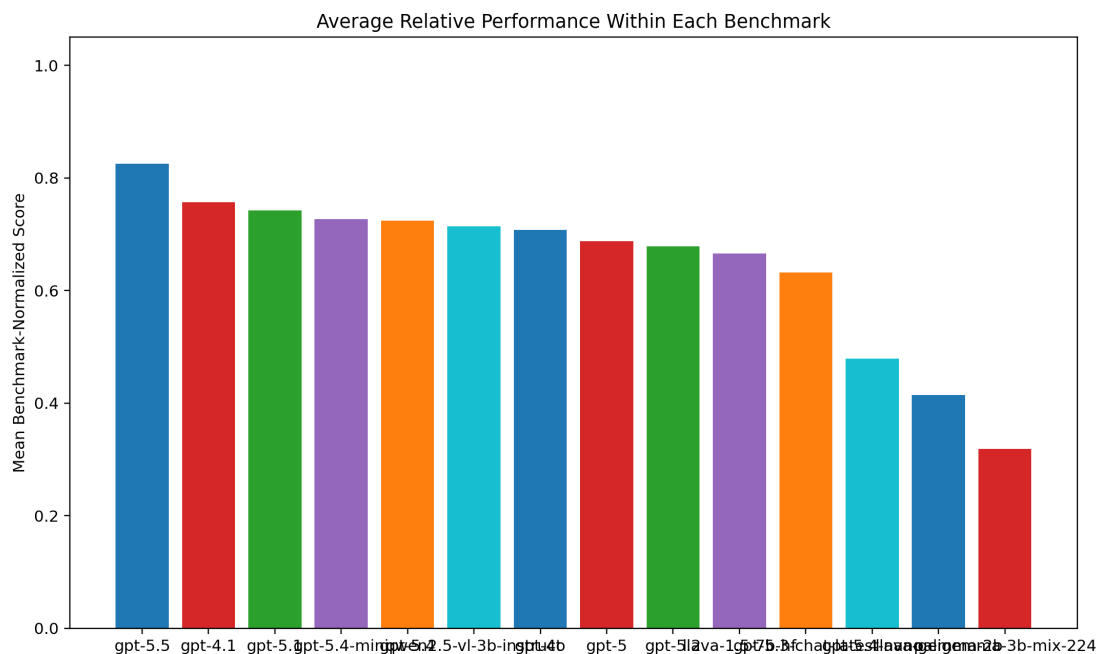


Figure 5.1: Mean score normalized within each benchmark. This is the primary cross-family ranking used in the report.

Table 5.1: Top ten models in the aggregate output.

Model	Normalized mean	Raw mean	Mean rank	Firsts
gpt-5.5	0.825	0.555	2.76	8
gpt-4.1	0.757	0.518	3.48	9
gpt-5.1	0.742	0.508	3.76	9
gpt-5.4-mini	0.727	0.496	4.48	7
gpt-5.4	0.724	0.505	4.81	6
qwen2.5-vl-3b-instruct	0.714	0.478	4.81	10
gpt-4o	0.708	0.495	4.62	6
gpt-5	0.688	0.499	4.81	8
gpt-5.2	0.678	0.480	5.19	6
llava-1.5-7b-hf	0.666	0.467	4.57	7

5.2 Raw scores give context, not a replacement leaderboard

Raw means tell a similar top-line story: **gpt-5.5** leads at 0.555, compared with 0.518 for **gpt-4.1**. That agreement is reassuring, but it does not make raw averaging enough. A classification task with values near one and a detection task with values near zero do not carry equal weight in a raw average. The raw chart is best used as a reality check before moving into the family-level analysis.

5.3 Winning often is not the same as winning broadly

The first-place count shows the most interesting tension in the table. Qwen2.5-VL 3B wins or ties for first on ten benchmarks, more than any other label, but it ranks sixth by normalized mean. In other words, it has real peaks and real weak spots. **gpt-5.5** has fewer first-place finishes, but is steadier across the full set.

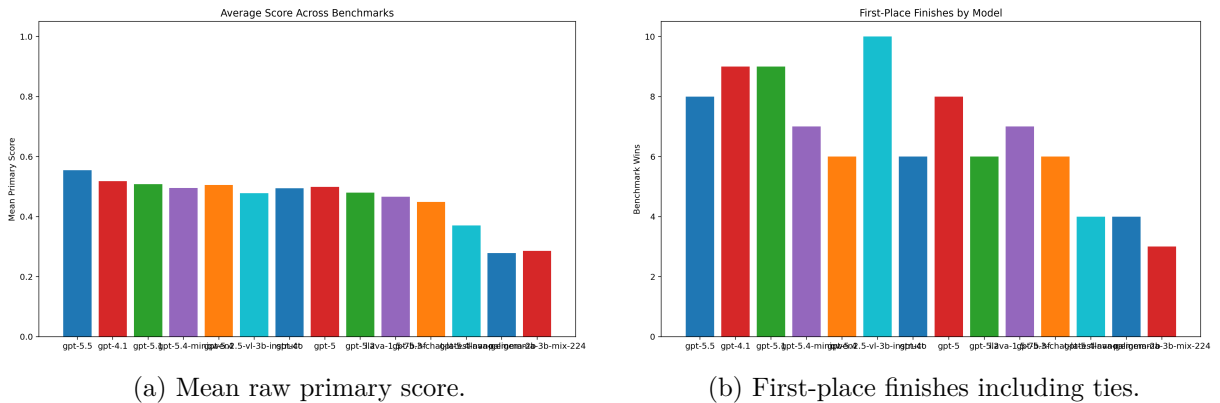


Figure 5.2: Two complementary aggregate views. Raw scores preserve original metric scale, while win counts reveal Qwen’s task-specific peaks.

5.4 Task-family comparison

5.4.1 Four families tell four different stories

The overall score is useful for a quick ranking, but the models make more sense once the tasks are separated. The family summary produces a different leader in almost every category. That is not noise; it tells us that captions, short answers, labels, and boxes stress the models in different ways.

Table 5.2: Family leaders by benchmark-normalized mean score.

Family	Leader or tied leaders	Normalized	Raw mean
Captioning	qwen2.5-vl-3b-instruct	0.977	0.325
Detection	gpt-5.4; gpt-5.5	0.873	0.159
Labeling	gpt-4.1; gpt-5.1	0.877	0.764
QA	paligemma-3b-mix-224	0.833	0.600

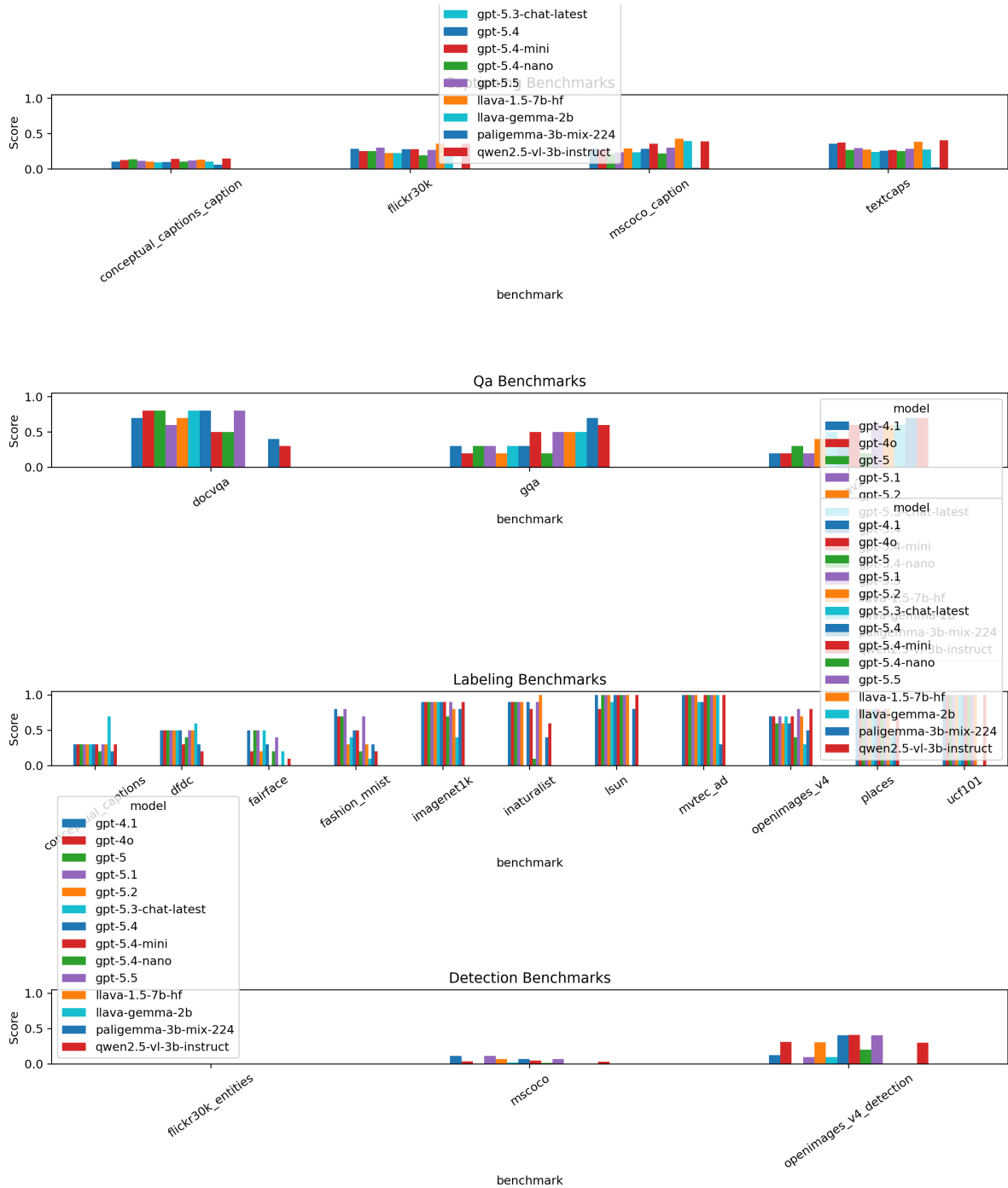


Figure 5.3: Scores separated into task families and component benchmarks. The panel view prevents a family average from concealing its source.

5.4.2 Captioning: local Qwen and LLaVA are competitive for a reason

Captioning is where Qwen looks strongest. `qwen2.5-vl-3b-instruct` gets a normalized family mean of 0.977. It wins three captioning rows: `conceptual_captions_caption`, `flickr30k`, and `textcaps`. `llava-1.5-7b-hf` is close behind at 0.936 and wins `mscoco_caption`. Their raw captioning means are almost identical after rounding, but normalization favors Qwen because it stays closer to the best result within the individual caption tasks.

For an application mostly built around descriptions, this matters more than the aggregate ranking.

Picking Qwen for captioning would not be a preference for a lower-ranked model; it would be following the more relevant evidence.

5.4.3 QA and labeling: an instructive reversal

PaliGemma is the best warning against dismissing a model from one aggregate number. It has the lowest normalized aggregate mean in the main table, 0.318, but it leads the normalized QA family summary at 0.833. At the same time, **gpt-5.5** has the higher raw QA mean, 0.633 versus PaliGemma’s 0.600, while landing behind it on normalized QA mean at 0.800.

That reversal is not a contradiction. The normalized summary gives each QA benchmark a relative voice; the raw average lets the original numeric scales matter more. Both facts belong in the report: **gpt-5.5** has the higher raw QA average here, while PaliGemma is the stronger relative competitor across the individual QA tasks after normalization.

Labeling is less surprising, but the top is tight. **gpt-4.1** and **gpt-5.1** tie for the normalized family lead at 0.877 and the raw family lead at 0.764. **gpt-5.5** follows closely at 0.864 normalized. For a label-heavy deployment, latency and operating constraints may matter more than these small score differences.

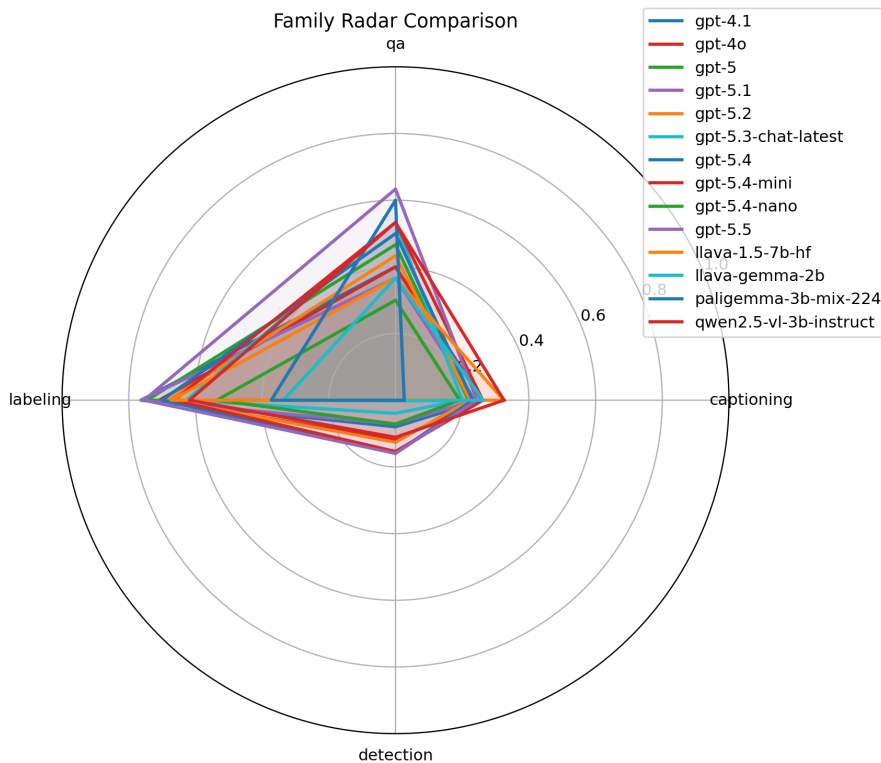


Figure 5.4: Family profile view. The chart is useful for seeing specialization, but exact conclusions should be read from the accompanying family table.

5.4.4 Detection: low values and a non-discriminating task

Detection sits on the lowest raw score scale of the four families. In the family mean, **gpt-5.4** and **gpt-5.5** tie at 0.873 normalized (raw 0.159), with **gpt-5.4-mini** close behind at 0.806 (raw 0.154). But this section needs a warning: **flickr30k_entities** gives every evaluated model a raw score of zero. That does not prove the models are equally bad at detection. It says this benchmark/protocol combination did not distinguish them on that row.

The other detection rows are more informative. On **mscoco**, **gpt-4.1** and **gpt-5.1** tie at 0.114. On **openimages_v4_detection**, **gpt-5.4-mini** leads at 0.413, just ahead of **gpt-5.4** and **gpt-5.5**, both at 0.405. Detection belongs in the report, but the all-zero row should be debugged before this part of the comparison is treated as settled.

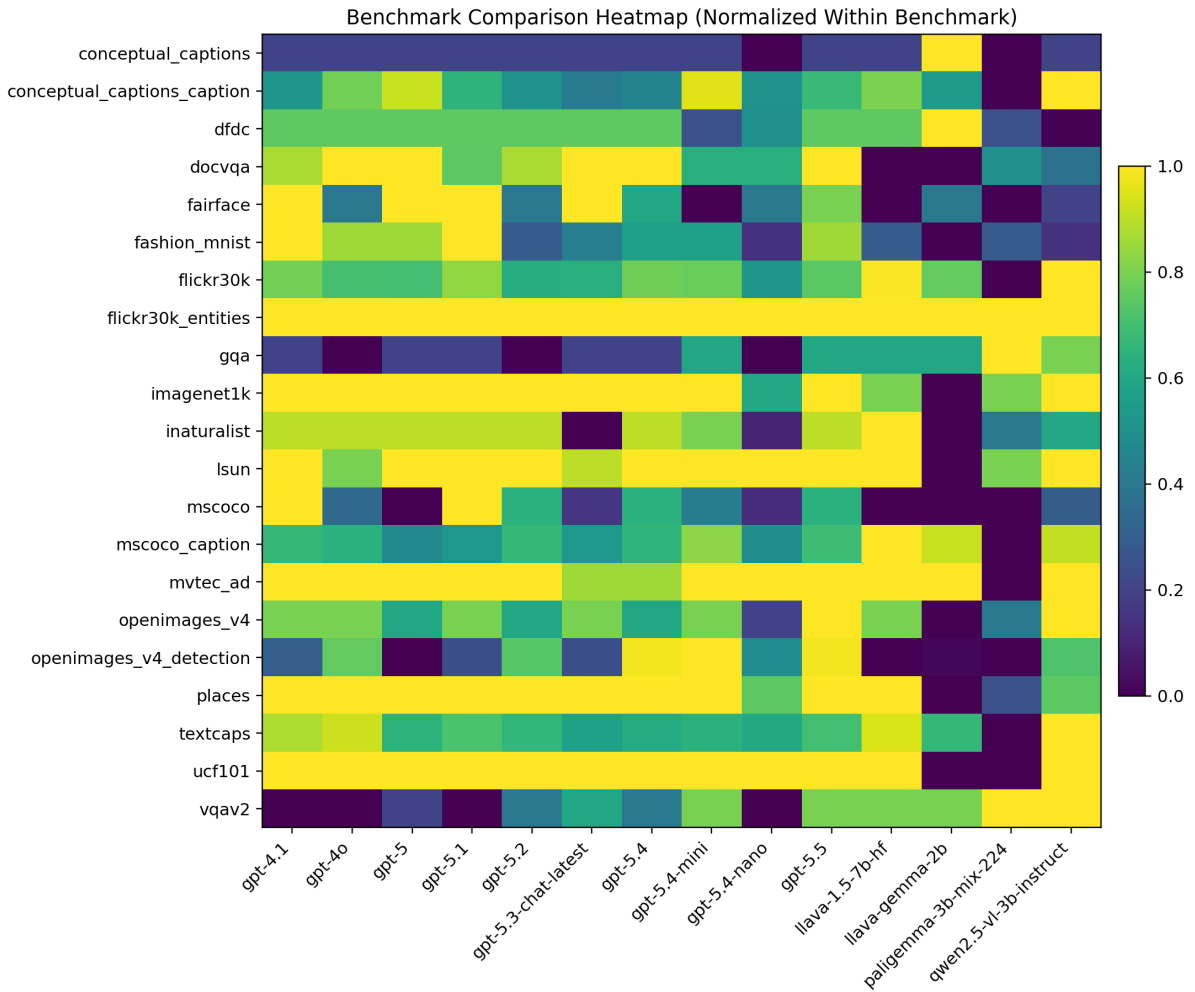


Figure 5.5: Normalized score heatmap. Bright cells expose specialist wins; flat or uniformly dark rows expose weak discriminatory evidence.

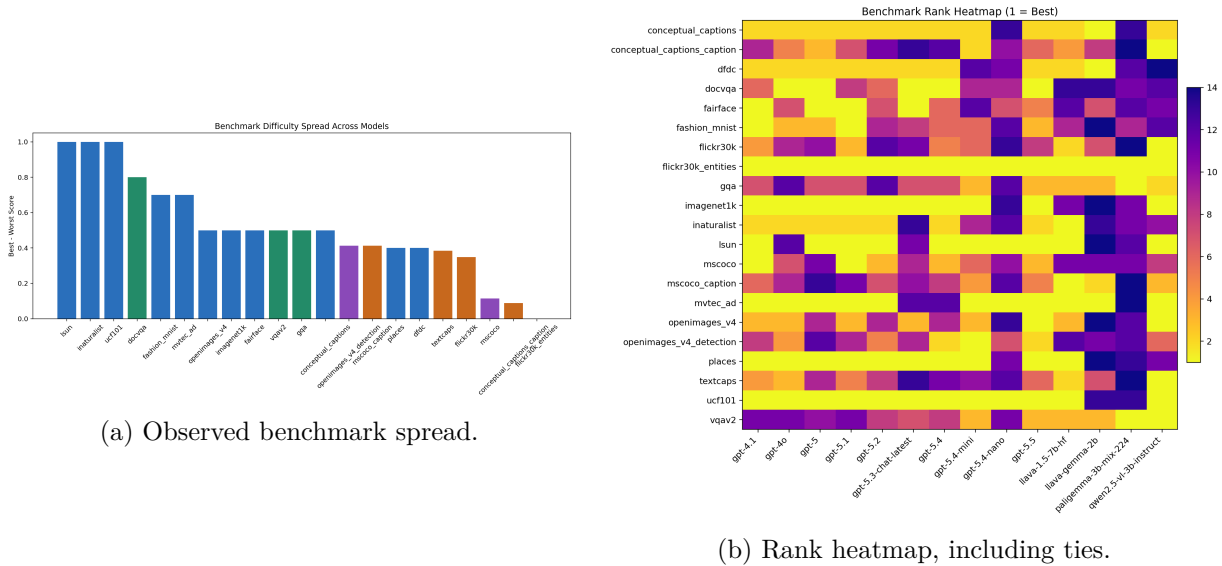
5.5 Spread, ranks, and uncertainty

5.5.1 Which benchmarks separate models?

Three benchmarks—`inaturalist`, `lsun`, and `ucf101`—span the complete observed range from 0.000 to 1.000. `docvqa` follows with a spread of 0.800, while `fashion_mnist` and `mvtec_ad` each span 0.700. These rows pull the eye in a heatmap, so they deserve extra attention. Before treating them as broad capability signals, it is worth checking the example counts and the actual mistakes.

5.5.2 Ranks retain ties that averages obscure

Rank views help where the score scales differ. They also expose tie-heavy rows, including tasks where many models reach the maximum. A first-place count should therefore be read as “reached the best recorded value,” not “uniquely solved the task.”



(a) Observed benchmark spread.

(b) Rank heatmap, including ties.

Figure 5.6: Spread and rank should be read together: separating tasks can also contain substantial ties or limited evidence.

5.5.3 Head-to-head comparisons are close at the top

The aggregate leader does not sweep its closest competitors. `gpt-5.5` beats `gpt-4.1` on seven benchmarks, loses on five, and ties on nine; its mean raw difference is only +0.037. Against `qwen2.5-vl-3b-instruct`, it wins eight, loses six, and ties seven, with a mean difference of +0.077. Those are real aggregate advantages, but not evidence that either alternative is generally unsuitable.

The bigger pairwise gaps appear farther down the table. `gpt-5.5` records 17 wins, no losses, and four ties against `gpt-5.4-nano`; against `palligemma-3b-mix-224` it records 18 wins, two losses, and one tie. Even that last comparison should be read alongside PaliGemma’s QA result.

5.5.4 Intervals make overstatement harder

The bootstrap intervals are generated by resampling benchmarks, not individual examples. They show how sensitive the raw mean is to the particular task mix in this report. `gpt-5.5` has observed

raw mean 0.555 with a 90 percent interval of [0.442, 0.665]. **gpt-4.1** is 0.518 with interval [0.401, 0.639], and **gpt-5.1** is 0.508 with [0.394, 0.631]. Those intervals overlap a lot.

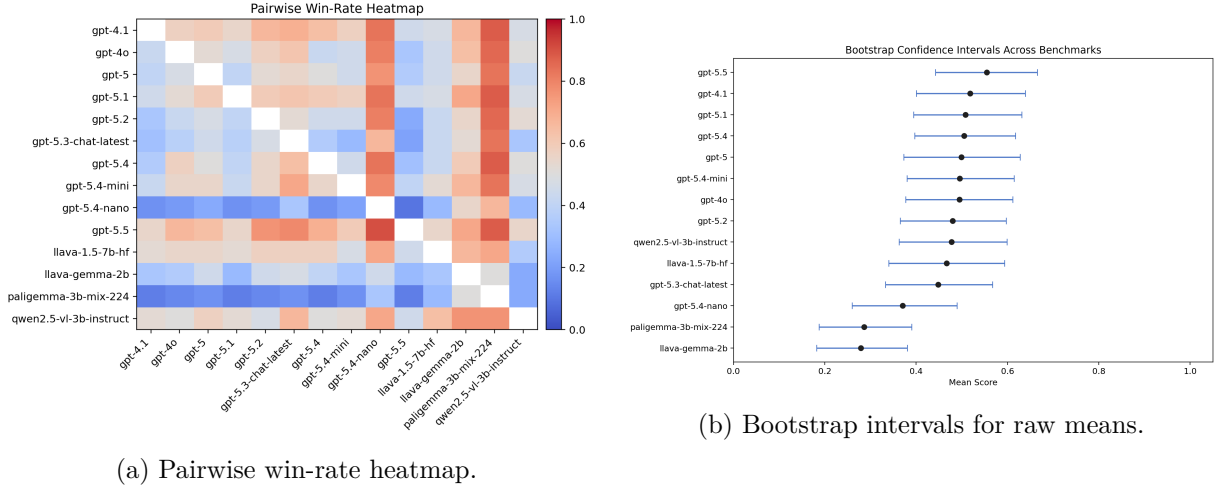


Figure 5.7: The mixed head-to-head pattern and overlapping intervals both caution against treating the top ordering as universal.

The fair reading is balanced. **gpt-5.5** leads this regenerated table by both raw and normalized summaries. At the same time, the result set is not large or uniform enough to promise the same order under a different application mix. In practice, the family plots and latency requirements should often make the final decision.

5.6 Runtime and resource trade-offs

5.6.1 Recorded timing

All main comparison cells report measured timing rather than estimated statistics. The fastest mean wall-clock values are **gpt-5.4-nano** (1.18 seconds), **llava-1.5-7b-hf** (1.19 seconds), and **gpt-5.4-mini** (1.21 seconds). The aggregate quality leader, **gpt-5.5**, averages 1.63 seconds. **qwen2.5-vl-3b-instruct** averages 2.70 seconds. PaliGemma and LLaVA-Gemma are much slower in this recorded table, at 12.81 and 99.07 seconds respectively.

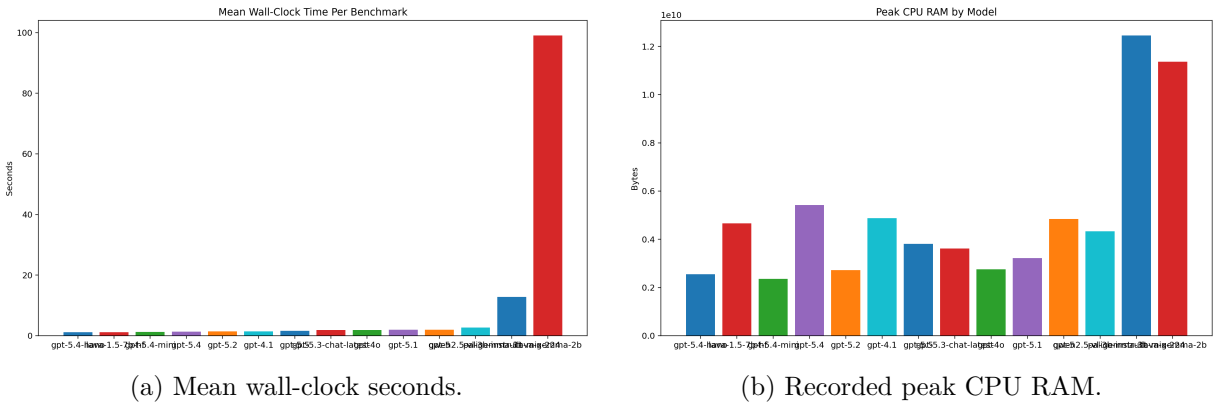


Figure 5.8: Runtime and memory measurements observed by the benchmark harness. Absolute values combine model pathway and execution environment.

5.6.2 A practical quality–latency frontier

If we prefer higher normalized score and lower wall-clock time, the observed non-dominated frontier contains five labels: `gpt-5.4-nano`, `llava-1.5-7b-hf`, `gpt-5.4-mini`, `gpt-4.1`, and `gpt-5.5`. Moving along that frontier buys more aggregate quality for modest increases in recorded time.

Qwen is not on that aggregate frontier because its 0.714 normalized score and 2.70-second mean are dominated in the all-task summary. It can still be the right choice for a caption-heavy application. The frontier uses the aggregate objective; Qwen’s strongest evidence is family-specific.

5.6.3 Memory values require environmental context

The memory chart is most useful for local runs, where model size and device constraints decide what is feasible. It is less useful for saying that one API-served system is intrinsically lighter than another. The repository measures its own process and observed call, not the provider’s serving infrastructure. The same caution applies to direct API-versus-local timing.

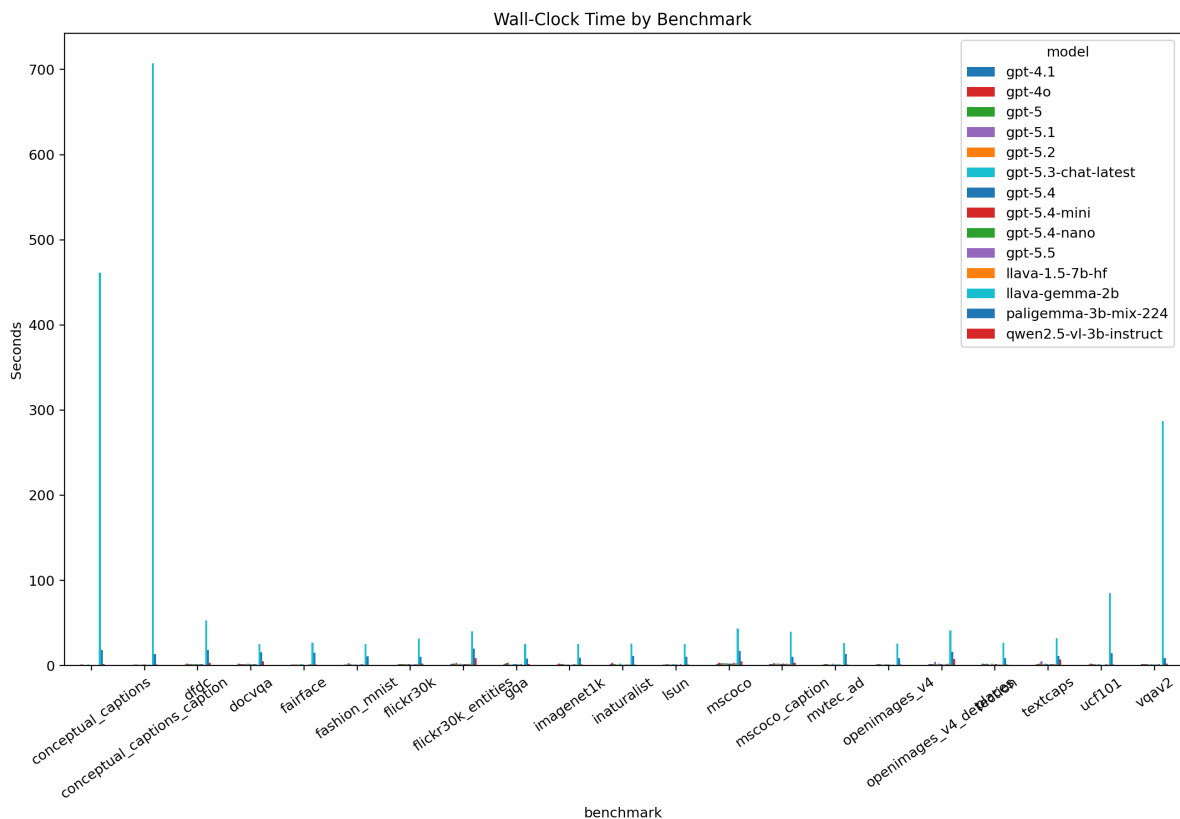


Figure 5.9: Wall-clock results separated by benchmark. Per-task variation can matter as much as a model mean when designing an interactive workflow.

Deployment implication. For a mixed-task workload in this measurement context, `gpt-5.5` gives the strongest score with moderate recorded time. For captioning, Qwen remains compelling despite slower aggregate timing. A production latency decision still needs repeated runs in the intended local or API environment.

Chapter 6

Fine-Tuning Experiments

6.1 Why a separate case study?

The main comparison asks how ready-to-use wrappers behave across task families. Fine-tuning asks a different question: can we change one model so it handles a narrow task better? For that reason, the fine-tuning result files stay outside the aggregate leaderboard and are analyzed as paired Fashion-MNIST evaluations.

Fashion-MNIST is a useful pilot because it is small, visual, and has ten clear classes. It also lets training data come from the training split while evaluation stays on the held-out test split. At the same time, it is not as easy as it looks for a language-generating vision model. Classes such as *shirt*, *pullover*, *coat*, and *t-shirt/top* are close enough visually that class collapse appears quickly.

6.2 QLoRA workflow

The generic script in `fine-tuning/train_qwen25vl_benchmark.py` reads manifest records with images, prompts, and target answers. The collator formats each example as a Qwen conversation and masks the prompt tokens so the loss is applied to the assistant answer rather than the instruction. The base model is loaded with 4-bit NF4 quantization and double quantization. LoRA modules are attached to the attention projections `q_proj`, `k_proj`, `v_proj`, and `o_proj`. The standard Qwen wrapper then loads the adapter for evaluation.

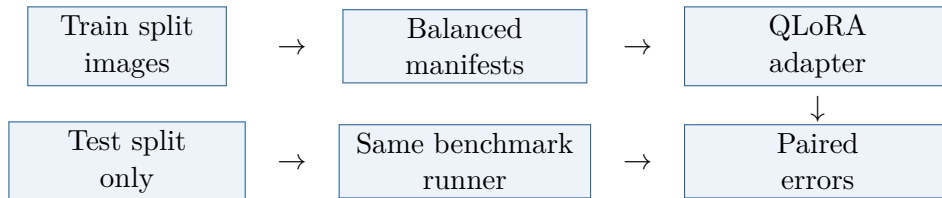


Figure 6.1: Fine-tuning and held-out evaluation workflow. The test examples do not enter the training manifests.

6.3 Accuracy improves, but the error distribution moves

The earlier LoRA run matches the base result exactly: 19 correct answers out of 50. The balanced LoRA run reaches 24 correct answers, a gain of five answers, or ten percentage points, on the same paired sample.

Table 6.1: Stored Fashion-MNIST Qwen2.5-VL-3B evaluation results.

Run	Correct	Total	Accuracy
Base test-split evaluation	19	50	38.0%
Earlier LoRA adapter	19	50	38.0%
Balanced LoRA adapter	24	50	48.0%

**Figure 6.2:** Accuracy comparison for the paired 50-example Fashion-MNIST evaluation.

The paired transition count says more than the bar chart. Compared with the base predictions, the balanced adapter changes 17 answers. Ten of those changes fix a base error. Five turn a previously correct answer into an error. The remaining changed answers move from one wrong label to another. So the adapter helps, but it does not improve the model uniformly.

Table 6.2: Correctness transitions from base to balanced adapter.

Transition	Count
Incorrect under base, correct under balanced adapter	10
Correct under base, incorrect under balanced adapter	5
Net gain in correct predictions	+5
All changed predictions	17

6.4 Class-level improvement and class bias

The adapter improves *shirt* sharply, from 2/5 correct examples to 5/5. It also fixes some *coat*, *trouser*, *ankle boot*, *bag*, and *sandal* examples. But the gain has a cost. *Pullover* drops from 4/6 to 1/6, and *t-shirt/top* drops from 2/3 to 0/3. Those losses are not random-looking: they sit in the same visually similar clothing region as the newly strengthened shirt label.

Table 6.3: Correct predictions by class: base versus balanced LoRA.

Class	Base	Balanced	Class	Base	Balanced
ankle boot	1/4	2/4	sandal	0/4	1/4
bag	2/4	3/4	shirt	2/5	5/5
coat	0/5	2/5	sneaker	7/7	7/7
dress	1/5	1/5	t-shirt/top	2/3	0/3
pullover	4/6	1/6	trouser	0/7	2/7

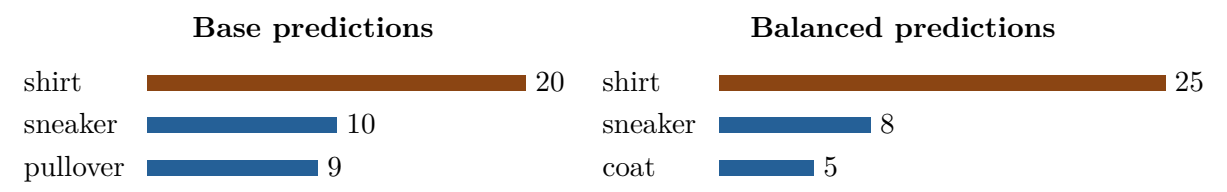


Figure 6.3: Most frequent predicted labels. The balanced adapter improves accuracy while increasing an already prominent preference for *shirt*.

There are only five true shirt instances in the fifty-example evaluation, yet the balanced adapter predicts shirt twenty-five times. That is the main caution in the case study. Balanced training improves measured accuracy, but it does not solve calibration among related garment classes. Before treating this adapter as more than a pilot, we would need a larger test, a confusion matrix, and repeated training seeds.

6.5 The Colab smoke result

A newer downloaded artifact contains a twenty-example base-versus-adapter smoke evaluation from the generalized Colab workflow. The base run gets 7/20 (35.0 percent), and the adapter run gets 9/20 (45.0 percent). Ten predictions change, five errors are fixed, and three correct base outputs regress.

That is useful operational evidence: the generalized training and evaluation path can produce an adapter, load it, and score it with the same runner. It is not strong additional performance evidence. Twenty examples are too few for a stable accuracy estimate, and this result should not be merged with the fifty-example comparison as if it were a planned replication.

Table 6.4: Interpretation of the two fine-tuning evidence sources.

Evidence source		Result	Suitable conclusion
Paired	50-example stored comparison	38.0% to 48.0%	Balanced LoRA improved this held-out pilot sample, with visible class bias.
Downloaded example	20-example Colab smoke pair	35.0% to 45.0%	The generalized Colab adapter/evaluation path runs end to end and can change measured outputs.

Fine-tuning conclusion. A narrow adapter can shift Qwen’s Fashion-MNIST behavior enough to improve the paired result. The new shirt bias is not a footnote; it is exactly the kind of error redistribution that has to be checked before calling a specialist model successful.

Chapter 7

Limitations and Next Experiments

7.1 Limits of the present evidence

The comparison is useful because it covers the full represented model–benchmark rectangle, but it is still a study of the artifacts available here. Some runs use modest sample sizes. Some rows have many ties. One detection row is zero for every model. Normalization helps combine unlike metrics, but it cannot turn a small or non-discriminating task into strong evidence.

The telemetry has a different limit. It honestly records what this harness observed, but the execution paths mix hosted API calls and local model runs. That can guide this repository’s workflow, but it cannot establish intrinsic inference efficiency across deployment modes.

The fine-tuning experiment is narrower again. It evaluates one adapter family on one classification task and one held-out sample. The accuracy gain is worth reporting, but the class-level regressions are exactly why more repetitions are needed.

7.2 Next experiments worth running

1. Repeat the balanced Fashion-MNIST QLoRA training with multiple seeds and larger held-out evaluations; report mean accuracy and confusion matrices.
2. Diagnose the all-zero `flickr30k_entities` detection result by checking prompt format, coordinate normalization, parsing, and example-level outputs before using it in model selection.
3. Fine-tune Qwen independently on one captioning task and one QA task. This tests whether its observed captioning strength can be enhanced without moving a narrow classifier’s bias problem into a new setting.
4. Record deployment mode and repeated latency trials explicitly, so the quality–speed comparison distinguishes API round-trip latency from local GPU generation.
5. Increase benchmark sample counts on rows with very large observed spread, then regenerate the bootstrap and pairwise analyses.

Reproduction Notes and Sources

Reproducing the comparison

The main tables and graphs in this report are generated from the top-level JSON reports under `results/`. From the repository root, run:

```
python comparison\build_report.py --results-dir results \
  --output-dir comparison\output --bootstrap-samples 5000
```

This produces the CSV tables and plots used in Chapters 4–7. The nested fine-tuning reports are intentionally not included by that command. They are examined separately so the baseline comparison stays separate from the task-adapted model.

Local sources used

File or directory	Role in this report
<code>comparison/output/model_summary.csv</code>	Aggregate normalized scores, raw means, ranks, and first-place totals.
<code>comparison/output/family_summary.csv</code>	Family-specific leader comparisons.
<code>comparison/output/pairwise_wins.csv</code>	Head-to-head benchmark outcomes.
<code>comparison/output/bootstrap_summary.csv</code>	Benchmark-resampled uncertainty intervals.
<code>comparison/output/telemetry_model_summary.csv</code>	Timing, memory, and measured-coverage summary.
<code>models/_openai_vision.py</code> and <code>models/_hf_model.py</code>	API and generic local inference pathways.
<code>models/_qwen2_5_vl.py</code> , <code>models/llava15_7b.py</code> , <code>models/paligemma_3b_mix_224.py</code>	Family-specific wrapper behavior discussed in Chapter 3.
<code>benchmarks/_base_benchmark.py</code>	Prompt, image, label, and evaluation contract.
<code>fine-tuning/train_qwen25vl_benchmark.py</code>	QLoRA configuration and manifest-driven training behavior.
<code>results/fine-tuning/*.json</code>	Stored 50-example Fashion-MNIST paired evaluations.

File or directory	Role in this report
<code>fine_tuning_smoke_artifacts(1)/*.json</code>	Downloaded 20-example Colab smoke comparison.

External architecture references

The architecture descriptions in Chapter 2 also use public model documentation for the local open model families. The API-served models are still described only at the request/response boundary visible in this repository.

Reference	Use in this report
https://huggingface.co/papers/2502.13923	Qwen2.5-VL technical report summary: dynamic-resolution vision transformer, window attention, and multimodal positional handling.
https://huggingface.co/docs/transformers/v4.51.3/en/model_doc/qwen2_5_vl	Transformers implementation documentation for Qwen2.5-VL conditional generation and configuration.
https://huggingface.co/docs/transformers/v4.48.1/en/model_doc/llava	Transformers LLaVA documentation: CLIP vision encoder, MLP projection, and autoregressive language-model generation.
https://ai.google.dev/gemma/docs/paligemma/model-card	Google PaliGemma model card: PaLI/Gemma lineage, SigLIP vision component, and Gemma language model component.

Closing observation

The main lesson is not simply that one model wins, and it is not that rankings are meaningless. The evaluated models do have an aggregate order in this result set, and `gpt-5.5` leads it. But the useful engineering decision usually happens one level lower: Qwen’s captioning behavior, PaliGemma’s QA result, the latency frontier, and the adapter’s shirt bias all change what “best” means for a concrete application. The value of the shared harness is that it makes those differences visible.